

#45 ログからのインシデント検出と自動改修の運用確立

状態	CLOSED
作成者	MITLabo
作成	2026-08-09 08:20 JST
最終更新	2026-08-09 09:37 JST
ラベル	(なし)
URL	https://github.com/shiftrepo/ai_workshop/issues/45

本文

markdown形式でプレゼン資料を作成するためのドキュメントを作ってほしい。また、一枚絵と詳細な複数枚絵も作成して、issueへのコメントとして追記する。対象のプロジェクトは下記である。デモの流れをベースとした資料としてください。 projects/onprem_log_training

コメント (2件)

コメント 1 — linear-code / 2026-08-09 08:20 JST

[SHI-51](#)

コメント 2 — MITLabo / 2026-08-09 09:00 JST

プレゼン資料とインフォグラフを作成しました

対象プロジェクト `projects/onprem_log_training` について、**デモの流れをベース**とした資料を作成しました。

- ブランチ: `MITLabo/issue-45` (コミット `dae26d8`)
- 想定読者: **マネジメント・意思決定層**
- スcope: **検知→収集を主軸**とし、自動改修は展望として扱っています

1. プレゼン資料 (Markdown)

[docs/04 incident detection presentation.md](#) — Marp形式・全25枚

GitHub上ではそのままMarkdownとして読めます。スライドに変換する場合:

```
npx @marp-team/marp-cli docs/04_incident_detection_presentation.md -o slides.html
npx @marp-team/marp-cli docs/04_incident_detection_presentation.md --pdf
```

構成:

内容	
導入	エグゼクティブサマリ / 現状の課題 / なぜ時刻突き合わせが破綻するか
打ち手	TrackIDによる相関 / これはツール導入ではなく運用定着
環境	研修環境の構成 / 題材「RoboMart」と仕込まれた不具合
デモの流れ	①バグを踏む ②検知しTrackID抽出 ③SSHで収集 ④1件に束ねる
整理	4段階分解 / AIエージェントに渡すべき前提 / Before-After / 工数の試算
展望	改修PRまで繋ぐ / ガードレール
実務	オンプレ制約 / 導入ステップ / リスクと対策 / まとめ

2. 一枚絵

全体を1枚に収めた概要図です。

ログからのインシデント検出

複数サーバのログを TrackID で1件に束ねる

なぜ障害調査に時間がかかるのか

① 「500エラーが出た」 — どのサーバの問題かまだ分らない

② アプリサーバのログを見る — エラーは見えるが原因は上流

③ 機能サーバに SSH して grep — **知っている人しか動けない**

④ 2つのログを時刻で目視突き合わせ — **推測が混じる**

⑤ チケットに手で転記

構造的な問題は **3 の属人化** と **4 の推測** による突き合わせ。
時刻の近さは同一処理の証明にならない。

打ち手 — 相関キー TrackID を通す

ブラウザ操作 在庫切れ商品の詳細ページを開く

TrackID を発行

client アプリサーバ層 → app.log に記録

HTTPヘッダ X-Track-Id: ABC1234

server 機能サーバ層 → service.log に記録

2つのログに **同じ TrackID** が残る
TrackID: **([A-Z0-9]{7})** で機械的に抽出できる

突き合わせが **推測** から **完全一致検索** になる = 自動化できる。

収集ツールは4段階に分けて作る

① 検知 app.log の ERROR 行を見つける

② 収集 SSH で service.log を TrackID 検索 — **接続の壁**

③ 関連付け TrackID をキーに1件へ集約 — **設計の壁**

④ 出力 一覧・レポート・通知として見せる

①②を先に動かし、通ってから③④を足す。
全部を一度に指示すると AI エージェントは的外れな実装に流れる。

運用として何が変わるか

BEFORE

- 時刻を目視、推測で突き合わせ
- SSH を知る人だけが調査できる
- 手でコピー転記
- 人は**探す作業**に時間を使う

AFTER

- TrackID で完全一致、機械的
- ガイドを読ませれば誰でも
- 1件のインシデントに自動整形
- 人は**判断**に集中する

本質は時間短縮ではなく「**探す作業**」を人の仕事から外したこと。
探すのは機械が得意、判断は人が得意 — 分担を正した。

この先の展望 — 改修 PR 発行まで繋ぐ

ログ収集済み自動 → 解析済み AI 一次解析 → 要承認 AI 改修案 → **PR作成待ち 人が承認** → 対応完了 PR 自動発行

承認ゲートは人が持つ。エージェントはブランチと PR まで — main へは直接反映しない。

研修環境: client(アプリ層 / ログはホストへ) + server(機能層 / SSH経由のみ) 不具合2件を常設 — 在庫切れページ 500 / 請求書払いの注文確定 500

projects/onprem_log_training

3. 詳細な複数枚絵

研修環境の構成 — 2つのサーバ役を、あえて非対称に作っている

研修環境の構成

2つのサーバ役を、あえて非対称に作っている

client

アプリケーションサーバ層

ブラウザからのアクセスを受け、画面と API を提供する

ポート 3002 (ブラウザから開く)

ログ client/logs/app.log

置き場所 ホストへ bind mount 済み

SSH 非搭載 — ホストから直接読める

内部 API 呼び出し

X-Track-Id: ABC1234

TrackID を下流へ引き渡す

server

機能(業務ロジック)サーバ層

在庫確認・注文金額の計算など、実際の業務ロジックを処理する

ポート 4002 (内部API) / 5101 (SSH)

ログ /app/logs/service.log

置き場所 コンテナ内のみ (mount しない)

SSH 搭載 — 収集練習の対象

なぜ非対称にしたか — 片方(client)は簡単に読め、片方(server)は SSH 経由でしか読めない。「手元で見えるログだけで満足するツール」では解けない構造を、意図的に作っている。

両方のログに同じ TrackID が残る

```
app.log (client / ホストから直接読める)
2026-07-25T09:00:00.123Z ERROR TrackID:ABC1234
[/api/robots/RBT-DOG-02] method=GET status=500
err=TypeError: Cannot read properties of undefined

service.log (server / SSH 経由のみ)
2026-07-25T09:00:00.089Z ERROR TrackID:ABC1234
[/internal/inventory/RBT-DOG-02] method=GET status=500
err=TypeError: Cannot read properties of undefined
```

両ログは同一フォーマット: <ISO8601> <LEVEL> TrackID:<7文字> [<パス>] key=value
時系列に注目 — 機能層の .089 がアプリ層の .123 より先。どちらが原因側か読める。

常設された2つの不具合

PRODUCT_STOCK_ZERO_NPE
再現: 在庫切れ商品「WalkyDog Mk2」の詳細ページを開く
原因: 在庫0の分岐で存在しないフィールドを .map() し TypeError
業務影響: 在庫切れページが 500 → 離脱率上昇

ORDER_TOTAL_UNDEFINED_TAX
再現: 決済方法「請求書払い」で注文確定
原因: 税率が未定義になり total.toFixed() で TypeError
業務影響: 法人向け請求書決済が全滅、機会損失甚大

server/bug-config.json の enabled で個別に ON/OFF。再現性が担保されているため、研修・リハールズで毎回同じ障害を起こせる。

起動: cd docker && ./setup-containers.sh start ブラウザ: http://localhost:3002/

docs/01_environment_overview.md

デモの流れ — バグ発火から、1件のインシデントに束ねるまで

デモの流れ

バグ発火から、1件のインシデントに束ねるまで

所要 1~2分

1 バグを踏む

演者がブラウザを操作する

操作: 在庫切れ商品「WalkyDog Mk2」の詳細ページを開く

結果: 画面に 500 エラー

この瞬間、2つのログに同じ TrackID が書かれる

```
app.log
ERROR TrackID:ABC1234
[/api/robots/RBT-DOG-02]
status=500

service.log
ERROR TrackID:ABC1234
[/internal/inventory/RBT-DOG-02] status=500
```

レスポンスにも track_id が含まれるため、演者は画面から追跡を始められる。

2 検知する

ツールが異常に気づく

AI エージェントに作らせたツールが app.log を監視し、ERROR 行を見つける。

正規表現で相関キーを抽出

TrackID:([A-Z0-9]{7})

↓

ABC1234

ここまでは簡単 — app.log はホストのローカルファイル。読むだけなら誰でもできる。

だが情報が足りない。アプリ層のログは「500 だった」しか語らない。原因は下流の機能サーバにある。ここで止まるツールが実務でいっぱい多い。

3 収集する

別サーバへ SSH で取りに行く

抽出した TrackID をキーに、機能サーバのログを検索する。

```
ssh -i training_key \
-p 5101 trainee@localhost \
"grep TrackID:ABC1234 \
/app/logs/service.log"
```

受講者はこのコマンドを覚えなない。接続先・ポート・鍵・ログの場所は AGENT_SSH_GUIDE.md に集約してあり、そのファイルを AI エージェントに読ませるだけでエージェント側が接続を組み立てる。

運用上の意味 — 「SSH が分かる人しか調査できない」という 属人化の解消。知識はドキュメントに置き、実行はエージェントに任せる。

4 1件に束ねる

人が読める記録にする

— INCIDENT —

TrackID : ABC1234

発生時刻: 09:00:00.089Z

概要 : 在庫切れ商品の詳細取得で TypeError

関連ログ:

```
[app.log .123]
ERROR /api/robots/... 500
[service.log .089]
ERROR /internal/... 500
```

時系列に注目 — 機能層の .089 がアプリ層の .123 より先。TrackID で束ねた瞬間に「どちらが原因側か」が読める。時刻の目視では出てこない情報。

この記録が、次の工程 (AI による一次解析 → 改修案 → PR 発行) の入力になる。

突き合わせが「推測」から「検索」に変わった

時刻の近さで判断するのをやめ、TrackID の完全一致で束ねる。人の勘に依存しないので、そのまま自動化できる。

調査の入口が属人化から外れた

接続情報はドキュメントに集約済み。SSH を知らない担当者でも同じ手順を回せる。

人の仕事「探す」から「決める」へ

①~③は機械が担い、人は④の判断だけを受け持つ。耐えるのは工数ではなく、判断までの待ち時間。

誰が担うか

受講者が組み立てるのは ①~④ のツール本体

① 検知

app.log の ERROR 行を見つける

ツールが自動

② 収集

SSH で service.log を TrackID 検索

AI エージェント

③ 関連付け

TrackID をキーに1件へ集約

ツールが自動

④ 出力 → 判断

記録を読み、対応方針を決める

人が担当

検知 → 収集 → 関連付け → 出力 の4段階のうち、①②を先に動かし、通ってから③④を足す

docs/04_incident_detection_presentation.md

AIエージェントへの指示の型 — 渡すべき6項目とプロンプト例

AI エージェントへの指示の型

研修の本当の成果物は、ツールではなく「この型を書ける担当者」

docs/02 が原典

なぜ「作って」だけでは通らないか

× 通らない指示

ログ収集ツールを作って

エージェントはこの環境固有の前提を知らない — このログを見るか、何をキーに結びつけるか、SSH でどこへ繋ぐか。
結果、的外れな実装になるか、毎回同じ質問をされて手戻りする。

✓ 通る指示

目的 / ログの場所 / 相関キー / 検知条件 / スcope / 出力形式
を最初にまとめて渡す

前提を先に渡せば、エージェントは一度で正しい方向へ進める。

渡すべき6項目 (チェックリスト)

- 目的 — client/server 2つのログを TrackID で紐づけ、1件のインシデントにする
- ログの場所 — client/logs/app.log (ローカル) と server 側 (SSH、AGENT_SSH_GUIDE.md を参照させる)
- 相関キー — TrackID: + 英数字7文字 / TrackID:([A-Z0-9]{7})
- 検知条件 — どの行を異常とみなすか (例: ERROR を含む行)
- スcope — 今回作るのは①②まで。段階的に依頼する
- 出力形式 — コンソール / ファイル / JSON

実行環境の制約も添える — SSH は鍵認証でパスワードなし、対話ログインではなくコマンド直接実行で十分。

プロンプト例 (そのまま使える)

```
# 目的
projects/onprem_log_training/ 配下に、client/server
2つのログを TrackID で関連付けるログ収集ツールを
作りたい。

# ログの場所
- client側: client/logs/app.log (ローカルファイル)
- server側: AGENT_SSH_GUIDE.md に書かれている
  SSH接続情報を使って取得すること

# 相関キー
両ログに共通で出現する TrackID:XXXXXX (英数字7文字)

# スcope (今回はここまで)
1. client側ログから ERROR 行を検知する
2. そのTrackIDでserver側ログを検索する
3. 両方のログをTrackID単位でまとめて表示する

# 今回は不要
常時監視の仕組み。1回実行して結果が出れば十分。
```

段階的に依頼する

① 検知
まずここだけ

② 収集
通ったら 次へ

③ 関連付け
後から 足す

④ 出力
最後に 整える

①②を先に動かし、通ってから③④を足す。最初から全部を一度に作らせようとすると、AI エージェントは的外れな実装に流れやすい。
常時監視 (常駐化) は最初のスcopeに入れない — Node.js で常駐させるか /loop で再実行させるかは、動いた後に受講者が選ぶ。

この型はログ収集に限らず適用できる — 目的・入力の場所・キー・スcope・出力を先に固定する

docs/02_building_log_collector_with_ai.md

この先の展望 — 改修PR発行まで繋ぐ

この先の展望 — 改修 PR 発行まで繋ぐ

検知・収集の次は「一次解析 → 改修案 → PR」。承認は人が持ち続ける

別プロジェクトで実装済み

1件のインシデントが辿る状態 — 進行役は Excel の1行

ログ収集済み
検知して自動起票
watchdog + log-summarizer

続行の判断
人が確認して進める
手動ゲート ⚙

解析済み
原因仮説・影響範囲
incident-analyzer

要承認
改修案・テスト計画
repair-planner

PR作成待ち
人が承認者を記入
手動ゲート ⚙

対応完了
PR URL が記録される
pr-publisher

止まる場所が2か所ある。起票された時 (収集ログと概要を見て「調査を続けるか」を決める) と、改修案が出た時 (内容を読んで「PRを出すか」を決める)。この2つは自動遷移しない — エージェントは人が進めるまで待つ。

誰が何を担うか

担い手	工程	やること
log_collector	収集	TrackID で client/server 両ログを集める (研修で作るのはここ)
log-summarizer	要約	生ログから事象の概要を1〜2文にする
人	続行判断	概要と収集ログを見て、調査を進めるかを決める
incident-analyzer	一次解析	症状・原因仮説・影響範囲・再現手順を出す (コードは書かない)
repair-planner	改修案	対象ファイル・変更方針・追加テスト・ロールバック手順を出す (コードは書かない)
人	承認	改修案をレビューし、承認者名を記入して PR 発行を許可する
pr-publisher	PR 発行	ブランチを切り、diff を適用し、Pull Request を出す

解析役と改修案役はコードを書き換えない。書き換えるのは最後の pr-publisher だけ — 変更を加える権限を1か所に絞ることで、レビュー対象が明確になる。

進行役を Excel の1行にしている理由

状態が目に見える
今日のインシデントがどこで止まっているかを、専用画面なしで一覧できる

承認の記録が残る
誰がいつ承認したかが行に残り、PR 本文にもそのまま転記される

人が割り込める
セルを書き換えれば進行を止める・戻すができる — 特別な操作を覚えなくてよい

まず着手するのは検知と収集 — 解析から先はこの土台の上に載る

前提としてのガードレール

- main へは直接反映しない。エージェントが出せるのはブランチと Pull Request まで。
- 承認ゲートは必ず人手。妙な改修案が出て PR 発行前に止まるため実害が出ない。
- テストが通らなければ Draft で出す。記録にも「テスト失敗」と明記し、そのまま流れない。
- 各エージェントにタイムアウト。応答が返らなければ「エラー」を書き戻して停止する。

1件が流れるまで — デモ台本上のタイムライン

00:00 バグ発火 → 検知 自動

00:05 収集・概要生成を終えて起票 自動

00:20 一次解析が書き戻される 自動

00:35 改修案が出て、ここで停止する 自動

03:30 Pull Request 発行・URL を記録 自動

研修環境をそのまま持ち出さないこと。平文の SSH 鍵と常時起動の sshd は学習用限定の割り切り。本番相当の環境では鍵管理・接続経路・権限を設計し直す前提で検討する。

projects/log_collector/auto-repair-demo/DEMO_FLOW.md

補足

- 画像は 2752×1536 (既存の onprem_log_training 資料と同じサイズ) です。
- 元データは docs/infographics_src/ のHTMLで、 ./render.sh で再生成できます。追加の依存パッケージはありません (ヘッドレスChromeのみ。日本語フォントの追加インストールも不要)。

- `./check-fit.sh` でレイアウトのはみ出しを検査できます。固定サイズで切り取られる都合上、あふれた内容はPNG上で黙って欠落するため、数値で検出できるようにしています。

資料作成時の判断について

- **工数の試算スライドの数値は実測値ではなく仮定値**です。スライド上にもその旨を明記しています。導入判断に使う際は自組織の実績値に置き換えてください。
- リスクのスライドには、本プロジェクトが元々ドキュメントに記載している注意 (**平文のSSH鍵と常時起動のsshdは学習用限定**、`main` へは直接反映しない) をそのまま引き継いでいます。